

Java

Inheritance, interface, abstraction, polymorphism, final , exception handling

Theory Notes

1. Abstraction

- **Definition:** Hiding implementation details and showing only essential features.
- Achieved using **abstract classes** and **interfaces**.
- Abstract class → can have abstract (incomplete) and non-abstract methods.
- Interface → full abstraction, only method declarations (Java 8+ allows default methods).

2. Exception Handling

- **Definition:** Mechanism to handle runtime errors gracefully.
- Keywords: try, catch, finally, throw, throws.
- Common exceptions: ArithmeticException, NullPointerException, ArrayIndexOutOfBoundsException, NumberFormatException.

3. Final Keyword

- final variable → constant, cannot be reassigned.
- final method → cannot be overridden.
- final class → cannot be inherited.

4. Inheritance

- **Definition:** Mechanism where one class acquires properties/methods of another.
- Types:
 - Single → parent → child.
 - Multi-level → parent → child → grandchild.
 - Hierarchical → one parent → multiple children.

- Hybrid/Multiple → achieved using interfaces.

5. Interfaces

- Provide **full abstraction**.
- A class can implement multiple interfaces → achieves multiple inheritance.
- Methods are implicitly public and abstract.

6. Polymorphism

- **Definition:** One name, many forms.
- Types:
 - Compile-time (method overloading).
 - Runtime (method overriding).

Coding Round Questions

1. Abstraction

- Create an abstract class Bank with abstract methods deposit() and withdraw().
- Implement them in a subclass SBI and demonstrate usage.

2. Exception Handling

- Write a program that takes a string input and converts it to an integer.
- Handle NumberFormatException if the input is invalid.

3. Final Keyword

- Create a class Circle with a final double PI = 3.14.
- Write a method to calculate area.
- Try modifying PI and observe the error.

4. Inheritance

- Demonstrate multi-level inheritance with classes Animal → Dog → Puppy.

- Each class should have its own method.

5. Interfaces

- Create two interfaces Printable and Showable.
- Implement both in a class Report and call their methods.

6. Polymorphism

- Write a program with a base class Payment and subclasses UPI and COD.
- Override the pay() method in each subclass.
- Demonstrate runtime polymorphism by storing subclass objects in a Payment reference.

Abstraction.java

```
abstract class abs{ // it doesn't allow for object creation
    // we won't achieve full abstraction because we have non-abstract method
    also
    abstract void absMethod(); //incomplete methods only function declaration
    abstract void store();
    void display(){
        System.out.println("Abstraction method class");
    }
}

class nonabs extends abs{

    void absMethod() {
        System.out.println("Abstraction method ");
    }
    void store(){
        System.out.println("Store method");
    }
}

public class Abstraction {
    public static void main(String[] args) {
        nonabs obj = new nonabs();
        obj.display();
        obj.absMethod();
        obj.store();
        // System.out.println("Abstraction");
        // abs obj = new abs(); //store in heap
        // stack => obj
    }
}
```

```

        // heap => object of abs
        // obj.display();
        // obj.absMethod();
    }
}

```

Inheritance.java

```

class Animal{
    void eat(){
        System.out.println("Animal will eat");
    }
}

//single inheritance
class Dog extends Animal{
    // void eat(){
    //     System.out.println("Dog will eat");
    // }
}
class Rat extends Dog{

}

//hierarchical inheritance
class Cat extends Animal{

}

public class InheritanceAbstractionPolyInterface {
    public static void main(String[] args) {
        // System.out.println("Hello there");

        // inheritance => single , multi-level, hierarchical, hybrid,
multiple
        Dog obj = new Dog();
        obj.eat();
        Cat obj1 = new Cat();
        obj1.eat();

        Rat obj2 = new Rat();
        obj2.eat();
    }
}

```

```

    }

    // parent => child //single
    // parent => multiple child //hierarchical
    // parent => child => grandchild //multi-level
    // hybrid and multiple inheritance achieved using interface
}

```

Interface.java

```

interface animal{
    void eat();
    void sound();
}

interface Bath{
    void water();
    void soap();
}

// multiple inheritance
class Tiger extends Elephant implements animal,Bath{

    public void eat(){
        System.out.println("Tiger eats deer");
    }

    public void sound(){
        System.out.println("Tiger sounds");
    }

    public void water(){
        System.out.println("Tiger bath with water");
    }

    public void soap(){
        System.out.println("Tiger bath without soap");
    }
}

// hybrid inheritance

class Elephant {
    void display(){
        System.out.println("Elephant display method");
    }
}

```

```

}
public class Interface {
    // interface => we can achieve full abstraction
    public static void main(String[] args) {

        Tiger obj = new Tiger();

        obj.water();
        obj.soap();
        obj.eat();
        obj.sound();
        obj.display();
    }
}

```

Polymorphism.java

```

class maths{
    // overloading => same method name,
    // but different in paramter size ,
    // and also importantly we can achieve in same class
    void add(int a, int b){
        System.out.println(a+b);
    }

    void add(int a, int b, int c){
        System.out.println(a+b+c);
    }
}

//overriding => runtime

class payment{
    void pay(){
        System.out.println("paid");
    }
}

class UPI extends payment{

    void pay(){
        System.out.println("upi");
    }
}

class COD extends payment{

```

```

        void pay(){
            System.out.println("cod");
        }
    }

    public class Polymorphism {

        public static void main(String[] args) {

            // UPI upi = new UPI();
            // upi.pay();

            // COD cod = new COD();
            // cod.pay();

            payment paid = new UPI();
            payment paid2= new COD();

            paid.pay();
            paid2.pay();

            maths obj = new maths();

            obj.add(10,5);
            obj.add(10,10,10);

            System.out.println("polymorphism");

            // polymorphism => poly -> many
            // same method
            // two types of polymorphism => compile-time, runtime polymorphism

            // compile -time => overloading
            // runtime => overriding
        }
    }
}

```

Exceptionhandling.java

```

public class ExceptionHandling {

    public static void main(String[] args) {

```

```

    int a =10;
    int b = 0;
    int c;
    String input = "123p";
    try{
        // int arr[] = null;
        // System.out.println(arr[0]);
        int number = Integer.parseInt(input);
        System.out.println(number);
        // int arr[]={1,2};
        // System.out.println(arr[10]);
        // c = a/b;
        // System.out.println(c);
    }catch(ArithmeticException e){
        System.out.println(e.getMessage());
    }catch(NullPointerException e){
        System.out.println(e.getMessage());
    }catch(ArrayIndexOutOfBoundsException e){
        System.out.println(e.getMessage());
    }catch(NumberFormatException e){
        System.out.println(e.getMessage());
    }catch(Exception e){
        System.out.println(e.getMessage());
    }finally{
        System.out.println("Finally passed through expections");
    }
    System.out.println("End");

}

}

```

Fina.java

```

final class finalized{

    final double PI = 3.14;
    final void display(){
        System.out.println("Final method");
    }
}

//final in field=> doesn't allow to modify the values
// final in method => doesn't allow to override
// final in class => doesn't allow to inherit

class inherited extends finalized{

```

```
    void display(){
        System.out.println("Inherited class");
    }
}

public class Final {

    public static void main(String[] args) {

        finaled obj = new finaled();

        // obj.PI =40;

        // System.out.println(obj.PI);

        inherited inherit = new inherited();

        inherit.display();

    }
}
```